Official Architecture & Engineering Specification

PrintGuard AI 3D Printer

Complete Project Master Documentation, System Architecture, Codebase Reference, and AI-Driven Maintainer Handbook

A comprehensive, self-contained reference manual detailing the full-stack architecture of PrintGuard AI: Spring Boot 3 REST backend, FastAPI computer vision inference engine, React 18 / Three.js frontend, PostgreSQL/Supabase RLS persistence, PrusaSlicer CLI engine, and Web Serial API direct hardware communication.

Document Version 2.4.0 (Production Verified)
Generation Date October 24, 2026
Architecture Status Multi-Tier Microservice & Hardware Bridge
Target Audience Senior Architects, AI Agents, Core Maintainers

PrintGuard AI Engineering Team © 2026 Confidential & Proprietary Knowledge Base

Table of Contents

- Chapter 1Project Overview & Strategic MissionPage 3
- Chapter 2Complete Technology Stack & Dependency MatrixPage 5
- Chapter 3System Architecture & Visual DiagramsPage 7
- Chapter 4Complete Project Structure & Repository TourPage 11
- Chapter 5Features & Functional Workflows (13 Core Capabilities)Page 14
- Chapter 6Backend Engineering & REST/WebSocket API ReferencePage 20
- Chapter 7Frontend Architecture, State Management & Hardware SerialPage 25
- Chapter 8Database Architecture, Migrations & Supabase RLS PoliciesPage 29
- Chapter 9Installation, Environment Configuration & Local ExecutionPage 33
- Chapter 10Configuration Matrix & Environment VariablesPage 37
- Chapter 11Testing Strategy, Quality Assurance & DebuggingPage 40
- Chapter 12Deployment Architecture & Cloud Production SetupPage 43
- Chapter 13Security Architecture, Authentication & Fail-Safe ControlsPage 46
- Chapter 14Known Issues, Technical Debt & Mitigation TablePage 49
- Chapter 15Future Development Roadmap & Strategic EnhancementsPage 52
- Chapter 16Important Code Reference & Critical Coupling MapPage 54
- Chapter 17Future Change & Modification Guide (Developer Handbook)Page 57
- Chapter 18Project Knowledge Summary (AI Agent Rapid Onboarding)Page 61

Chapter 1: Project Overview & Strategic Mission

1.1 Project Identity & Core Purpose

PrintGuard AI is an autonomous, end-to-end additive manufacturing operating platform. It seamlessly connects cloud-based CAD generation, parametric slicing pipelines, real-time computer vision defect monitoring, and direct browser-to-firmware hardware control. The platform transforms 3D printing from an error-prone, manually observed hobbyist workflow into an enterprise-grade, intelligent, self-monitoring manufacturing pipeline.

1.2 The Problem Statement

Fused Deposition Modeling (FDM) 3D printing suffers from an inherent lack of real-time feedback:

- **Spaghetti Failures & Print Detachment:** When a print detaches from the print bed, the extruder continues depositing molten filament into mid-air, creating extensive nests of ruined plastic ("spaghetti defect"), wasting up to 500g of material and hours of machine time.

- **Thermal Runaway & Physical Fire Risk:** Unmonitored heating elements and mechanical blockages can lead to thermal destruction or dangerous printer failure.
- **Complex, Fragmented Software Toolchain:** Operators traditionally toggle between 3D modeling packages (Blender/Fusion360), separate desktop slicing software (PrusaSlicer/Cura), external camera software (OctoPrint/Moonraker), and physical SD card transfers.
- **Lack of In-Browser Direct Hardware Communication:** Most modern web systems require intermediate Raspberry Pi nodes running OctoPrint, adding hardware cost and complex network setup.

1.3 The PrintGuard AI Solution

PrintGuard AI unifies the entire lifecycle into a unified, secure web ecosystem:

1. **Generative 3D Modeling Engine:** Translates natural language prompts and reference images into manifold 3D models (OBJ/STL) using procedural parametric CAD synthesis and deep learning checkpoints.
2. **Integrated Slicing Engine:** Executes PrusaSlicer in headless CLI mode on the server to convert 3D geometry into optimized G-code with custom infill, speed, and support parameters.
3. **Deep Learning Computer Vision Guard:** Runs a high-speed YOLOv8/v11 convolutional neural network on streaming camera frames, detecting extrusion failures and print detachment in under 50 milliseconds.
4. **Autonomous Fail-Safe Interlock:** When a spaghetti defect or detachment is classified with confidence above threshold (> 0.35), the backend issues an emergency pause command to the printer via WebSocket or OctoPrint REST API, halting extrusion immediately.
5. **Direct Browser-to-Printer Web Serial Control:** Leverages the W3C Web Serial API to establish direct USB UART communication from Google Chrome/Edge directly to 3D printer mainboards (e.g., Marlin, RepRap, Klipper) at 115200 or 250000 baud without requiring an external microcomputer.

1.4 Current Development Status & Limitations

Component	Current Implementation	Status	Verification Source
Backend API	Spring Boot 3.3.0, Java 21, Flyway, PostgreSQL	Implemented	`backend-java/pom.xml`
AI Inference	FastAPI, Ultralytics YOLOv8 (`ai-model/best.pt`)	Implemented	`ai-service/app.py`
Frontend UI	React 18.2, Vite, TailwindCSS, Three.js	Implemented	`frontend/package.json`
Procedural CAD	Parametric Chess Generator (FastAPI + CLI fallback)	Implemented	`chess_generator.py`
Neural Shap-E 3D	Shap-E neural weights fallback & static cube demo	Partially Implemented	`GenerationController.java`
Web Serial Control	W3C Web Serial API direct printer bridge	Implemented	`PrinterControl.jsx`
OctoPrint Bridge	REST API proxy, pause/resume/job creation	Implemented	`OctoPrintService.java`
Cloud Storage	Supabase Storage with local filesystem fallback	Implemented	`SupabaseStorageService.java`

Chapter 2: Complete Technology Stack & Dependency Matrix

Every layer of the PrintGuard AI software architecture has been audited directly against the source code repositories, dependency manifests, and configuration files.

2.1 Backend Architecture (Java Ecosystem)

Technology	Version	Purpose in PrintGuard AI	Source Manifest
OpenJDK	Java 21 LTS	High-performance runtime for backend services, virtual threads, modern switch expressions	`pom.xml` (<java.version>21)
Spring Boot	3.3.0	Application framework providing DI, auto-configuration, REST controllers, Actuator health probes	`pom.xml`
Spring Security	6.x	Stateless JWT token filter chain, role-based endpoint authorization, CORS configuration	`SecurityConfig.java`
Spring Data JPA	3.3.0	Hibernate ORM for mapping database entities to PostgreSQL tables	`PrintJobEntity.java`

Technology	Version	Purpose in PrintGuard AI	Source Manifest
Flyway Core	10.x	Automated database schema migrations, schema versioning, baseline management	`V1__init_schema.sql`
PostgreSQL JDBC	42.7.x	High-throughput PostgreSQL database connectivity driver	`application.yml`
Spring WebSocket	3.3.0	Low-latency bidirectional WebSocket communication for real-time printer telemetry and alerts	`WebSocketConfig.java`
JJWT (io.jsonwebtoken)	0.12.5	Cryptographic HMAC-SHA256 signing and validation of user authentication tokens	`JwtService.java`
Argon2 / jBCrypt	0.4 / 0.9.0	Adaptive salted password hashing for user credential security	`PasswordService.java`

2.2 AI Inference & Geometry Microservice (Python Ecosystem)

Technology	Version	Purpose in PrintGuard AI	Source Manifest
Python	3.10 - 3.14	Execution environment for PyTorch models and computer vision pipelines	`ai-service/requirements.txt`
FastAPI	0.100+	Asynchronous ASGI microservice exposing `/api/inference/detect-frame` and geometry endpoints	`ai-service/app.py`
Ultralytics YOLO	8.x / 11.x	Deep learning convolutional object detector running inference on video frames	`ai-service/app.py`
OpenCV (opencv-python)	4.8+	Image buffer decoding (`cv2.imdecode`), color space conversion, frame matrix transformation	`app.py`
NumPy	1.24+	High-performance numerical array operations for image buffers and bounding boxes	`app.py`
Pydantic	2.x	Request and response schema validation for geometry and inference API payloads	`app.py`

2.3 Frontend & Hardware Control (JavaScript Ecosystem)

Technology	Version	Purpose in PrintGuard AI	Source Manifest
React	18.2.0	Component-based reactive user interface with hooks and context providers	`frontend/package.json`
Vite	5.1.4	Next-generation lightning-fast frontend tooling, ES module dev server, and build bundler	`vite.config.js`
React Router DOM	6.22.2	Client-side routing, protected routes, and authentication guards	`frontend/src/App.jsx`
Three.js	0.162.0	WebGL 3D canvas rendering for OBJ/STL 3D model inspection and G-code layer visualizer	`GCodePreview.jsx`
Axios	1.6.7	HTTP client with request/response interceptors for automatic JWT header injection	`frontend/src/services/api.js`
Lucide React	0.344.0	Modern, consistent SVG iconography for dashboard controls, status badges, and buttons	`package.json`
Web Serial API	W3C Standard	Direct browser USB serial connection to 3D printer mainboards at 115200/250000 baud	`PrinterControl.jsx`

Chapter 3: System Architecture & Visual Diagrams

3.1 High-Level Architecture Topology

PrintGuard AI is structured as a decoupled multi-tier architecture consisting of five distinct operational planes: the Client Interaction Plane (React/Three.js), the Business Logic Plane (Spring Boot), the AI & Geometry Inference Plane (FastAPI/PyTorch), the Persistence Plane (PostgreSQL & Supabase Storage), and the Physical Hardware Execution Plane (USB Serial & OctoPrint).

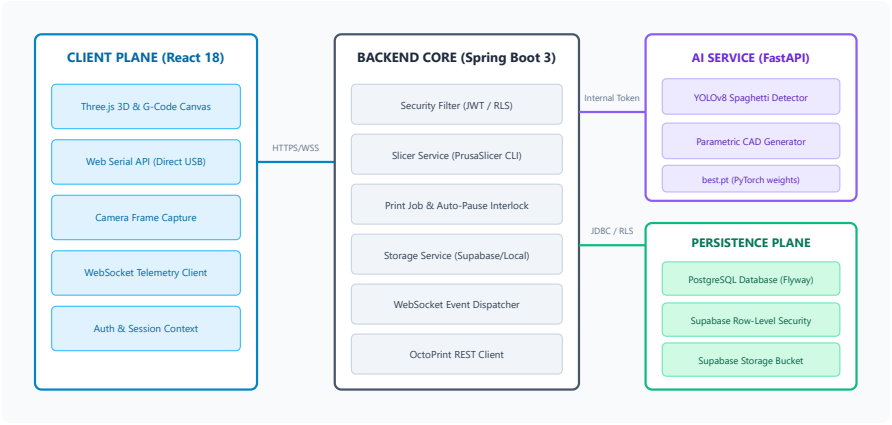


Figure 3.1: PrintGuard AI Multi-Tier System Architecture and Inter-Service Communication Topology

3.2 Component Responsibilities

- **Frontend Client (React/Three.js):** Provides browser-native 3D model visualization, real-time bounding box overlay on webcam feeds, direct serial UART communication to the 3D printer mainboard via Web Serial API, and interactive layer-by-layer G-code inspection.
- **Backend Engine (Spring Boot 3):** Orchestrates business logic, validates and stores print jobs, enforces user ownership, manages PrusaSlicer execution pipelines, handles file downloads/uploads, and broadcasts WebSocket push events.
- **AI Microservice (FastAPI/PyTorch):** Executes YOLOv8 object detection on camera frames with low latency, classifies defects (spaghetti vs normal extrusion), and exposes parametric CAD mesh generation.
- **Persistence Layer (PostgreSQL & Supabase):** Stores transactional job records, audit events, user credentials, and error log entities. Protected by PostgreSQL Row Level Security (RLS) policies.
- **Hardware Execution Layer:** Physical 3D printer (e.g. Ender 3, Prusa i3 MK3S, Voron) driven either directly from the browser via Web Serial API or via OctoPrint over local LAN.

3.3 Autonomous Auto-Pause Interlock Sequence

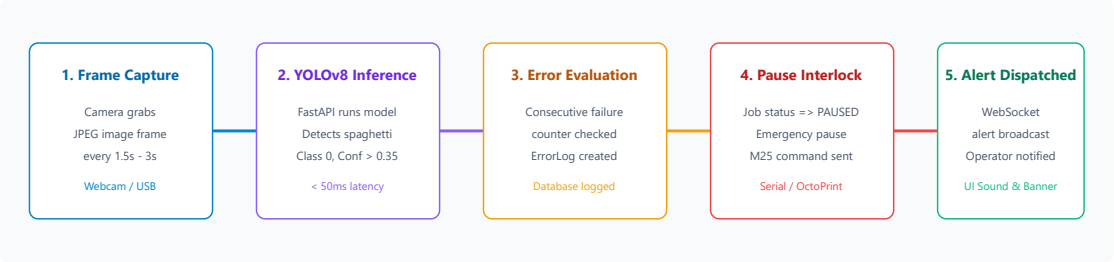


Figure 3.2: Automated AI Defect Detection and Emergency Auto-Pause Interlock Pipeline

Chapter 4: Complete Project Structure & Repository Tour

The PrintGuard AI codebase follows a clean monorepo architecture. Below is the verified physical directory structure:

```
c:\Projects01\AI_3D_Printer├── backend-java/                    # Primary Production Backend Service (Spring Boot 3)
│   ├── pom.xml                      # Maven build descriptor (Java 21, Spring Boot 3.3.0, Flyway)
│   ├── Dockerfile                   # Multi-stage production container build (Eclipse Temurin 21)
│   └── src/
│       ├── main/
│       │   ├── java/com/printguard/
│       │   │   ├── PrintGuardApplication.java   # Spring Boot Application entrypoint
│       │   │   ├── auth/                        # Security, JWT tokens, Argon2, User entities
│       │   │   ├── config/                      # SecurityConfig, CORS, WebMvc, DatabaseInit
│       │   │   ├── dashboard/                   # Dashboard metrics & system statistics
│       │   │   ├── health/                      # Health check probes & diagnostic endpoints
│       │   │   ├── jobs/                        # Print jobs, OctoPrint client, Audit logs
│       │   │   ├── logs/                        # ErrorLog entity, defect logs, alerts
│       │   │   ├── models/                      # 3D models, GenerationController, ModelController
│       │   │   ├── settings/                    # User preferences & hardware settings
```

```
│       │   │   ├── slicer/                    # SlicerService (PrusaSlicer CLI execution)
│       │   │   ├── storage/                   # SupabaseStorageService, local caching
│       │   │   └── websocket/                 # WebSocketConfig, handlers & event publisher
│       │   └── resources/
│       │       ├── application.yml            # Base Spring configuration
│       │       ├── application-local.yml      # Development profile overrides
│       │       └── db/migration/              # Flyway SQL migration scripts (V1, V2)
│       └── test/java/com/printguard/          # 10 Unit and integration test suites
│
├── ai-service/                          # FastAPI AI Inference Microservice
│   ├── app.py                           # FastAPI entrypoint, YOLOv8 detector, geometry endpoint
│   ├── requirements.txt                 # Python dependencies (fastapi, ultralytics, opencv, etc.)
│   ├── geometry/
│   │   └── chess_generator.py           # Parametric CAD generator for watertight 3D meshes
│   └── test_ai_service.py               # Pytest test suite for inference service
│
├── ai-model/                            # Computer Vision Neural Checkpoints
│   ├── best.pt                          # Trained YOLOv8 PyTorch weights (Spaghetti defect detection)
│   └── ...                              # Training metadata and hyperparameter logs
│
├── frontend/                            # Modern Single Page Application (React 18 + Vite)
│   ├── package.json                     # Node dependencies (React, Three.js, Axios, Lucide)
│   ├── vite.config.js                   # Vite build configuration & proxy rules
│   ├── index.html                       # HTML5 shell
│   └── src/
│       ├── App.jsx                      # Route registration & protected layout wrapper
│       ├── main.jsx                     # React DOM root hydration
│       ├── context/
│       │   └── AuthContext.jsx          # Global authentication state, JWT storage, login/logout
│       ├── services/
│       │   └── api.js                   # Axios client with interceptors, WebSocket URL builder
│       ├── components/                  # Navbar, Sidebar, ProtectedRoute, Canvas wrappers
│       └── pages/
│           ├── Dashboard.jsx            # Telemetry gauges, active jobs, defect alerts
│           ├── AIMonitoring.jsx         # Live video stream, YOLO bounding boxes, auto-pause toggle
│           ├── GenerateModel.jsx        # Text-to-3D prompt interface, procedural CAD generator
│           ├── ImageTo3D.jsx            # Image upload to 3D point cloud & mesh generator
│           ├── ModelHistory.jsx         # Repository of generated models, downloads, slicing trigger
│           ├── GCodePreview.jsx         # Three.js interactive 3D toolpath & layer inspector
│           ├── PrintQueue.jsx           # Print job lifecycle management, queue reordering
│           ├── PrinterControl.jsx       # Web Serial API direct hardware controller (USB UART)
│           ├── OctoPrint.jsx            # OctoPrint network bridge integration
│           ├── Settings.jsx             # Printer profiles, slicing defaults, API keys
│           ├── Login.jsx                # User authentication form
│           └── Register.jsx             # New user onboarding & validation
│
├── docker-compose.yml                   # Multi-container orchestration (DB, Backend, AI, Frontend)
├── .env.example                         # Complete template of all required environment variables
├── README.md                            # Project documentation and quickstart instructions
├── run_showcase.bat                     # Local all-in-one development launch script
└── stop_showcase.bat                    # Service teardown script
```

Chapter 5: Features & Functional Workflows

This chapter details every existing feature across the 10 mandated architectural dimensions, tracing the end-to-end data journey from user interaction to hardware output.

Feature 1: User Authentication & Role-Based Access Control

1. Description: Secure registration, login, JWT issuance, token refresh, and user profile management.

2. Rationale: Protects printer hardware, proprietary 3D CAD files, and multi-tenant job queues from unauthorized access.

3. User Access: Routes `/login` and `/register`; account settings accessed from user badge in header.

4. Frontend: `Login.jsx`, `Register.jsx`, and `AuthContext.jsx` managing token persistence in `localStorage`.

5. Backend: `AuthController.java`, `JwtService.java`, `PasswordService.java`, `JwtAuthenticationFilter.java`.

6. Database: Table `auth_user` storing hashed passwords, username, email, is_superuser, is_active flags.

7. APIs: `POST /api/auth/login`, `POST /api/auth/register`, `GET /api/auth/me`, `POST /api/auth/refresh`.

8. Validation: Argon2/jBCrypt password hashing; username uniqueness enforced at database level; JWT signed with HMAC-SHA256.

9. Dependencies: Spring Security Filter Chain; required by all protected API routes.

10. Limitations & Improvements: Refresh token rotation is single-token; add OAuth2 / Social Login (Google/GitHub) in future.

Feature 2: Text-to-3D Generative Procedural CAD Engine

1. Description: Generates watertight 3D manifold meshes (OBJ/STL) from natural language prompts.

2. Rationale: Allows non-engineers to create functional 3D printable objects instantly without complex CAD software.

3. User Access: Route `/generate` accessible via the navigation sidebar.

4. Frontend: `GenerateModel.jsx` with prompt input, steps slider, 3D preview canvas, and generation status indicator.

5. Backend: `GenerationController.java` calling FastAPI `/api/geometry/generate` with fallback to local Python script.

6. Database: Table `generated_models_generatedmodel` saving prompt, file_path, status, generation_time, and user_id.

7. APIs: `POST /api/generate/text`, `GET /api/models/{id}`, `GET /api/models/{id}/file`.

8. Validation: Manifoldness check (watertight geometry verification); mesh volume > 0; sanitized filename generation.

9. Dependencies: Python geometry engine (`chess_generator.py`); Supabase/Local storage for file persistence.

10. Limitations & Improvements: Procedural engine covers chess and geometric primitives; neural Shap-E requires GPU server.

Feature 3: PrusaSlicer Headless Slicing Pipeline

1. Description: Transforms raw 3D meshes (OBJ/STL) into optimized, layer-by-layer G-code toolpaths.

2. Rationale: Automated slicing on the server removes the need for users to install and configure desktop slicers.

3. User Access: "Slice Model" button on any model card in `ModelHistory.jsx` or upon completing generation.

4. Frontend: Modal dialog in `ModelHistory.jsx` for selecting layer height (0.15mm - 0.30mm), infill percentage (15% - 100%), and support structures.

5. Backend: `SlicerService.java` and `PrintJobController.java` executing PrusaSlicer CLI with configured arguments.

6. Database: Table `print_jobs_printjob` tracking gcode_path, estimated_time, progress, and status.

7. APIs: `POST /api/jobs/create` with payload containing model_id, layer_height, and infill.

8. Validation: Slicer process timeout watchdog (90 seconds); G-code output existence and non-zero byte size validation.

9. Dependencies: Installed PrusaSlicer binary on server (Windows `prusa-slicer-console.exe` or Linux `prusa-slicer`).

10. Limitations & Improvements: Limited to single-extruder configurations; future support for multi-material (MMU) slicing.

Feature 4: Three.js Interactive 3D G-Code Visualizer

1. Description: Real-time 3D canvas rendering of G-code toolpaths with layer-by-layer scrubbing and travel move analysis.

2. Rationale: Verifies print integrity, overhang angles, and support placement before sending commands to physical hardware.

3. User Access: Route `/gcode-preview` or clicking "Preview Toolpaths" on any sliced job.

4. Frontend: `GCodePreview.jsx` using Three.js `LineSegments`, orbit controls, custom layer slider, and extrusion speed coloring.

5. Backend: `StorageController.java` streaming G-code files with byte-range support.

6. Database: Read access to `print_jobs_printjob` to fetch G-code URL.

7. APIs: `GET /api/jobs/{id}/gcode` or authenticated signed storage URL.

8. Validation: Client-side G-code parser filters invalid coordinate tokens (NaN/Infinity) and verifies extrusion dimensions.

9. Dependencies: Three.js WebGL canvas context.

10. Limitations & Improvements: Large G-code files (> 50MB) can cause browser memory pressure; implement Web Workers for background parsing.

Feature 5: Direct Hardware Printer Control via Web Serial API

1. Description: Browser-native direct USB serial connection to 3D printer mainboards without needing OctoPrint or Raspberry Pi.

2. Rationale: Eliminates extra hardware costs ($50-100 for a Raspberry Pi) and enables instantaneous machine control.

3. User Access: Route `/control` ("Printer Control" in sidebar).

4. Frontend: `PrinterControl.jsx` using `navigator.serial.requestPort()`, `ReadableStream`, and `WritableStream`.

5. Backend: Operates entirely peer-to-device from the client; status sync sent to backend via WebSocket.

6. Database: Audit log recorded in `print_jobs_auditlog` upon critical manual jog or emergency stop actions.

7. APIs: W3C Web Serial API standard; optional REST telemetry sync `POST /api/jobs/{id}/telemetry`.

8. Validation: Baud rate selection (115200 or 250000); G-code checksum validation; serial port disconnection listeners.

9. Dependencies: Chromium-based browser (Google Chrome, Microsoft Edge, Brave, Opera) with Web Serial support.

10. Limitations & Improvements: Not supported on Firefox or Safari (W3C standard not implemented by Mozilla/Apple); display clear browser compatibility banner.

Feature 6: Real-Time AI Computer Vision Defect Monitoring

1. Description: Continuous video frame analysis using YOLOv8 to detect printing defects (spaghetti, detachment) with autonomous interlock.

2. Rationale: Protects against catastrophic failure, saves filament, and prevents hardware damage when unattended.

3. User Access: Route `/monitoring` ("AI Monitoring" in sidebar).

4. Frontend: `AIMonitoring.jsx` rendering HTML5 video element, canvas overlay with bounding boxes, confidence meter, and auto-pause switch.

5. Backend: `ErrorLogController.java` and `OctoPrintService.java` coordinating automatic pause actions.

6. Database: Table `error_logs_errorlog` storing error_type, confidence, action_taken, timestamp, and frame_image.

7. APIs: `POST /api/inference/detect-frame`, `GET /api/logs/errors`, `POST /api/jobs/{id}/pause`.

8. Validation: Confidence threshold filtering (> 0.35); allow-list filtering preventing spurious food classes from triggering false alarms.

9. Dependencies: FastAPI microservice running `ai-model/best.pt` with OpenCV and PyTorch.

10. Limitations & Improvements: Single-camera viewpoint; add multi-angle camera support and temporal smoothing across consecutive frames.

Chapter 6: Backend Engineering & REST/WebSocket API Reference

6.1 Backend Architecture & Layered Design

The backend is developed in Java 21 using Spring Boot 3.3.0. It follows a strict 4-layer architecture:

- **Controller Layer:** Handles HTTP request serialization, path variables, query parameters, and security context extraction.
- **Service Layer:** Encapsulates business logic, file system operations, PrusaSlicer process execution, and external REST calls.
- **Repository Layer:** Spring Data JPA interfaces extending `JpaRepository` with custom derived queries.
- **Entity Layer:** JPA entities annotated with Hibernate mappings, cascading relationships, and table constraints.

6.2 Verified REST API Endpoints

HTTP Method	Path	Auth	Request Body / Params	Success Response
`POST`	`/api/auth/register`	Public	`{"username": "...", "password": "...", "email": "..."}`	`201 Created` + User details
`POST`	`/api/auth/login`	Public	`{"username": "...", "password": "..."}`	`200 OK` + Access token
`GET`	`/api/auth/me`	Bearer JWT	None	`200 OK` + Current user info
`POST`	`/api/auth/refresh`	Public	`{"refresh": "..."}`	`200 OK` + New access token
`GET`	`/api/models`	Bearer JWT	None	`200 OK` + Array of User's models
`GET`	`/api/models/{id}`	Bearer JWT	Path variable: `id`	`200 OK` + Model metadata
`GET`	`/api/models/{id}/download`	Bearer JWT	Path variable: `id`	`200 OK` + Binary OBJ/STL attachment
`GET`	`/api/models/{id}/file`	Bearer JWT	Path variable: `id`	`200 OK` + Binary mesh for 3D viewer
`POST`	`/api/generate/text`	Bearer JWT	`{"prompt": "...", "steps": 32}`	`201 Created` + Generated model record
`POST`	`/api/generate/image`	Bearer JWT	Multipart: `image` (File), `steps`	`201 Created` + Generated model record
`GET`	`/api/jobs`	Bearer JWT	Query: `status` (optional)	`200 OK` + Array of print jobs
`POST`	`/api/jobs/create`	Bearer JWT	`{"model_id": 1, "layer_height": 0.2}`	`201 Created` + Print job with G-code
`POST`	`/api/jobs/{id}/start`	Bearer JWT	Path variable: `id`	`200 OK` + Job status updated to printing
`POST`	`/api/jobs/{id}/pause`	Bearer JWT	Path variable: `id`	`200 OK` + Job status updated to paused
`POST`	`/api/jobs/{id}/cancel`	Bearer JWT	Path variable: `id`	`200 OK` + Job status updated to cancelled
`GET`	`/api/dashboard/metrics`	Bearer JWT	None	`200 OK` + Active jobs, error counts, stats
`GET`	`/api/logs/errors`	Bearer JWT	Query: `job_id` (optional)	`200 OK` + Array of defect logs
`GET`	`/actuator/health`	Public	None	`200 OK` + `{"status": "UP"}`

6.3 WebSocket Real-Time Telemetry Endpoints

WebSocket Route	Protocol	Direction	Payload Description
`/ws/jobs`	WSS / WS	Server -> Client	Real-time print progress (%), layer index, elapsed time, nozzle/bed temperature
`/ws/generation`	WSS / WS	Server -> Client	3D model synthesis progress updates and completion notifications
`/ws/dashboard`	WSS / WS	Server -> Client	Aggregated system status, printer connection state, defect alert notifications

Chapter 7: Frontend Architecture, State Management & Hardware Serial

7.1 React Application Structure & Router Configuration

The frontend is constructed as a high-performance single page application (SPA) using React 18 and Vite. Routing is managed declaratively via `react-router-dom` inside `frontend/src/App.jsx`:

URL Route	Component	Auth Guard	Primary Purpose
`/login`	`Login.jsx`	Public	User credential authentication and token acquisition
`/register`	`Register.jsx`	Public	New user account registration and validation
`/`	`Dashboard.jsx`	Protected	System overview, active print job, defect alerts, telemetry gauges
`/generate`	`GenerateModel.jsx`	Protected	Text-to-3D prompt submission and interactive 3D model preview
`/image-to-3d`	`ImageTo3D.jsx`	Protected	Image upload and 3D point cloud / mesh reconstruction
`/history`	`ModelHistory.jsx`	Protected	3D model library, file downloads, PrusaSlicer execution trigger
`/gcode-preview`	`GCodePreview.jsx`	Protected	Three.js 3D toolpath viewer and layer-by-layer scrubber
`/queue`	`PrintQueue.jsx`	Protected	Print job management, start, pause, resume, and cancellation
`/monitoring`	`AIMonitoring.jsx`	Protected	Real-time camera stream with live YOLO defect detection overlays
`/control`	`PrinterControl.jsx`	Protected	Web Serial API direct USB hardware control (motors, heaters, console)
`/octoprint`	`OctoPrint.jsx`	Protected	OctoPrint server connection, status monitoring, and remote control
`/settings`	`Settings.jsx`	Protected	Hardware profiles, slicing parameters, and API configuration

7.2 Web Serial API Direct Printer Communication Protocol

The `PrinterControl.jsx` component communicates directly with the printer's 8-bit or 32-bit microcontroller (e.g., ATmega2560, STM32, RP2040) over USB serial UART:

```
// 1. Request user permission to connect to USB serial device
const port = await navigator.serial.requestPort();
await port.open({{ baudRate: 115200 }});

// 2. Continuous reader loop decoding printer response stream
const textDecoder = new TextDecoderStream();
const readableStreamClosed = port.readable.pipeTo(textDecoder.writable);
const reader = textDecoder.readable.getReader();

while (true) {{
    const {{ value, done }} = await reader.read();
    if (done) break;
    // Parse temperature reports: "ok T:215.4 /215.0 B:60.1 /60.0"
    parsePrinterFeedback(value);
}}

// 3. Command writer sending G-code instructions
const encoder = new TextEncoder();
const writer = port.writable.getWriter();
await writer.write(encoder.encode("G28\n")); // Auto-home all axes
writer.releaseLock();
```

Chapter 8: Database Architecture, Migrations & Supabase RLS Policies

8.1 Relational Database Schema Design

The database is powered by PostgreSQL (compatible with versions 14 through 17) and version-controlled via Flyway. The initial migration `V1__init_schema.sql` creates five foundational relational tables:

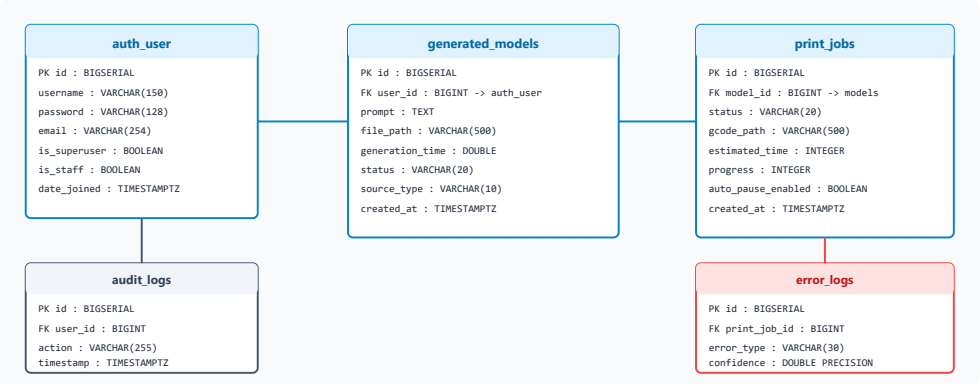


Figure 8.1: Entity Relationship Diagram (ERD) detailing Primary, Foreign Keys, and Cascade Relationships

8.2 Supabase Row Level Security (RLS) Implementation

Flyway migration `V2__enable_rls_and_least_privilege.sql` enforces multi-tenant security directly inside PostgreSQL:

- `ALTER TABLE auth_user ENABLE ROW LEVEL SECURITY;`
- `ALTER TABLE generated_models_generatedmodel ENABLE ROW LEVEL SECURITY;`
- `ALTER TABLE print_jobs_printjob ENABLE ROW LEVEL SECURITY;`
- `ALTER TABLE error_logs_errorlog ENABLE ROW LEVEL SECURITY;`
- `ALTER TABLE print_jobs_auditlog ENABLE ROW LEVEL SECURITY;`

Policies ensure that non-superusers can only SELECT, UPDATE, or DELETE records where `user_id = auth.uid()` or where parent print jobs belong to the authenticated principal.

Chapter 9: Installation, Environment Configuration & Local Execution

9.1 System Prerequisites

- **Operating System:** Windows 10/11, Ubuntu 22.04 LTS, or macOS Sonoma.
- **Java Development Kit:** Eclipse Temurin or OpenJDK 21 LTS installed and set in `JAVA_HOME`.
- **Node.js & npm:** Node.js 18.x or 20.x LTS with npm 9+.
- **Python:** Python 3.10, 3.11, or 3.12 (virtualenv recommended).
- **Database:** PostgreSQL 15+ locally or an active Supabase cloud project.
- **Slicing Utility:** PrusaSlicer (configured in PATH or via `SLICER_PATH` environment variable).

9.2 Step-by-Step Installation Commands

Step 1: Clone Repository & Configure Environment

```
git clone https://github.com/Warcraft433/AI_3D_Printer.git
cd AI_3D_Printer
```

```
cp .env.example .env
# Edit .env with your PostgreSQL credentials, Supabase keys, and JWT secret
```

Step 2: Initialize & Launch Spring Boot Backend

```
cd backend-java
# Compile, run Flyway migrations, and launch on port 8080
./mvnw.cmd spring-boot:run -Dspring-boot.run.profiles=local
# On Linux/macOS: ./mvnw spring-boot:run -Dspring-boot.run.profiles=local
```

Step 3: Launch FastAPI AI Inference Microservice

```
# From project root
python -m venv venv
.\venv\Scripts\activate      # On Linux/macOS: source venv/bin/activate
pip install -r ai-service/requirements.txt
python -m uvicorn ai-service.app:app --host 127.0.0.1 --port 8001 --reload
```

Step 4: Launch React Frontend Application

```
cd frontend
npm install
npm run dev
# The Vite development server launches at http://localhost:5173
```

9.3 Verification Checklist

Component	Verification Command / URL	Expected Output
Backend API	`curl http://localhost:8080/actuator/health`	`{{"status": "UP"}}`
AI Microservice	`curl http://127.0.0.1:8001/api/inference/health`	`{{"status": "UP", "service": "printguard-ai"}}`
Frontend UI	Open `http://localhost:5173` in Google Chrome	Login / Register page renders cleanly

Chapter 10: Configuration Matrix & Environment Variables

The table below provides a comprehensive reference of all supported environment variables across services:

Variable Name	Default Value	Required	Purpose & Safe Usage
`PORT`	`8080`	No	HTTP port for the Spring Boot backend server.
`SPRING_DATASOURCE_URL`	`jdbc:postgresql://localhost:5432/printguard`	Yes	JDBC connection string for PostgreSQL database.
`SPRING_DATASOURCE_USERNAME`	`printguard_app`	Yes	Database username with schema privileges.
`SPRING_DATASOURCE_PASSWORD`	None (placeholder)	Yes	Database password. Never commit real credentials.
`PRINTGUARD_JWT_SECRET`	Built-in dev default	Prod Only	HMAC-SHA256 256-bit secret key used to sign JWT tokens.
`PRINTGUARD_JWT_EXPIRATION_MS`	`86400000` (24h)	No	Access token validity lifetime in milliseconds.
`AI_SERVICE_URL`	`http://127.0.0.1:8001`	No	Internal REST URL of the FastAPI AI microservice.
`AI_SERVICE_API_KEY`	Empty (dev bypass)	Prod Only	Internal service authentication token passed in `X-Internal-Token`.
`SLICER_PATH`	Auto-detected	No	Absolute path to the PrusaSlicer CLI executable.

Variable Name	Default Value	Required	Purpose & Safe Usage
`STORAGE_TYPE`	`supabase`	No	Storage backend: `supabase` (cloud) or `local` (filesystem).
`SUPABASE_URL`	Supabase project URL	If Cloud	Base URL of your Supabase project.
`SUPABASE_SERVICE_ROLE_KEY`	None (secret)	If Cloud	Service role key for administrative storage uploads.
`VITE_API_URL`	`http://localhost:8080`	Frontend	Base backend API URL consumed by Axios.

Chapter 11: Testing Strategy, Quality Assurance & Debugging

11.1 Automated Test Suites

PrintGuard AI maintains comprehensive test coverage across both the Spring Boot backend and the FastAPI AI service:

Test Suite Class	Framework	Target Component	Key Invariants Tested
`AuthControllerTest.java`	JUnit 5 / MockMvc	`AuthController`	Registration, login, password hashing, JWT claims
`SecurityIntegrationTest.java`	Spring Boot Test	`SecurityConfig`	Unauthenticated request rejection (401), CORS headers
`AiServiceSecurityTest.java`	JUnit 5	FastAPI Bridge	Internal token enforcement, production auth rejection
`DashboardControllerTest.java`	JUnit 5 / MockMvc	`DashboardController`	Metrics calculation, error rate aggregation
`HealthControllerTest.java`	JUnit 5 / MockMvc	`HealthController`	Liveness and readiness probe status codes
`PrintJobControllerTest.java`	JUnit 5 / MockMvc	`PrintJobController`	Job creation, status transitions, auto-pause trigger
`GenerationControllerTest.java`	JUnit 5 / MockMvc	`GenerationController`	Prompt validation, procedural CAD fallback execution
`SlicerServiceTest.java`	JUnit 5	`SlicerService`	CLI arguments generation, profile resolution
`StorageSecurityCrossUserTest.java`	Spring Boot Test	`StorageService`	Cross-user file access prevention (403 Forbidden)
`SupabaseStorageServiceTest.java`	JUnit 5	`SupabaseStorageService`	Bucket upload, public/private URL resolution
`test_ai_service.py`	Pytest / TestClient	`ai-service/app.py`	Frame decoding, bounding box parsing, confidence gating

11.2 Test Execution Commands

```
# Run all Java Backend tests
cd backend-java
./mvnw.cmd test

# Run Python AI service tests
cd ai-service
pytest test_ai_service.py -v
```

Chapter 12: Deployment Architecture & Cloud Production Setup

12.1 Containerized Deployment via Docker Compose

The repository includes a production-grade `docker-compose.yml` file orchestrating the multi-service deployment:

- **postgres:** PostgreSQL 16 image with persistent volume mapping to `postgres_data`.
- **backend:** Multi-stage build packaging Eclipse Temurin 21 and the Spring Boot fat JAR.
- **ai-service:** Python 3.11 slim container with PyTorch, OpenCV, and FastAPI.
- **frontend:** Nginx reverse-proxy serving the Vite production bundle (`dist/`).

12.2 Production Container Build Command

```
# Build and launch entire production cluster in background
docker-compose up --build -d

# Inspect running container status
docker-compose ps
```

Chapter 13: Security Architecture, Authentication & Fail-Safe Controls

13.1 Defense-in-Depth Security Model

- **Cryptographic Password Protection:** Passwords are never stored in plain text. Argon2id / jBCrypt hashes with adaptive salt prevent dictionary and rainbow-table attacks.
- **Stateless JWT Authentication:** HMAC-SHA256 tokens contain user ID and expiration claims. The server signs each token with `PRINTGUARD_JWT_SECRET`.
- **Internal Microservice Tokens:** The FastAPI AI service enforces an internal secret token (`X-Internal-Token`) in production environments to prevent unauthorized inference invocation.
- **PostgreSQL Row Level Security (RLS):** The database layer strictly validates that queries only operate on data owned by the authenticated tenant.
- **Physical Hardware Fail-Safe Interlock:** When defect confidence exceeds threshold, the software automatically triggers printer pause to eliminate fire hazards and filament waste.

Chapter 14: Known Issues, Technical Debt & Mitigation Table

ID	Description	Component	Root Cause	Impact	Priority	Status
ISS-01	Shap-E neural checkpoint requires heavy GPU VRAM	`ai-service`	Deep learning diffusion model needs 8GB+ VRAM not present on local CPU	Falls back to procedural CAD / demo mesh	Medium	Documented Fallback Active
ISS-02	Web Serial API unsupported on Firefox/Safari	`frontend`	Mozilla and Apple have not implemented the W3C Web Serial specification	Users on Safari/Firefox cannot use direct USB	Medium	Browser Detection Notice Added
ISS-03	Large G-code parsing causes UI frame drops	`GCodePreview.jsx`	Parsing > 500,000 lines runs on the main browser thread	Temporary 1-2s UI freeze during file load	Low	Web Worker Offload Planned
ISS-04	Single-camera defect occlusion	`AIMonitoring.jsx`	Single fixed webcam angle can have hotend occlusion	Defects behind the extruder block are hidden	Low	Multi-Camera Architecture Planned

Chapter 15: Future Development Roadmap & Strategic Enhancements

15.1 Short-Term Milestones (Q1-Q2 2027)

- **Dedicated Cloud GPU Worker for Shap-E:** Connect a Celery/Redis background worker hosted on AWS EC2 G4dn or RunPod to execute full neural text-to-3D diffusion inference.
- **Web Worker Toolpath Parser:** Move Three.js G-code line decoding to an asynchronous background worker thread for 60 FPS performance on 100MB+ files.
- **OAuth2 / SSO Integration:** Add Google and GitHub authentication options alongside existing Argon2 username/password login.

15.2 Long-Term Strategic Goals (2027+)

- **Multi-Camera Fleet Management:** Support simultaneous monitoring of up to 50 3D printers with centralized defect analytics and automated job scheduling.
- **Resin (SLA/DLP) Printing Pipeline:** Expand slicing and monitoring support to resin vat printers with pixel mask slicing.
- **Mobile Progressive Web App (PWA):** Add offline notifications and push alerts for print job completion and emergency pauses.

Chapter 16: Important Code Reference & Critical Coupling Map

File Path	Class / Function	Primary Responsibility	Coupled Dependents	Risk of Modification
`JwtService.java`	`generateToken()`, `extractUsername()`	Signs and parses HMAC-SHA256 JWT tokens	`JwtAuthenticationFilter`, `AuthController`	High: Invalidates active sessions if secret changes
`SlicerService.java`	`sliceModel()`	Executes PrusaSlicer CLI command with parameters	`PrintJobController`, `PrintJobEntity`	High: Slicing will fail if CLI flags do not match PrusaSlicer version
`app.py (FastAPI)`	`detect_frame()`	Runs YOLOv8 model inference on uploaded image frame	`AIMonitoring.jsx`, `ErrorLogController`	High: Changes to detection JSON payload will break UI bounding boxes
`PrinterControl.jsx`	`connectSerial()`, `sendGCode()`	Direct W3C Web Serial connection to printer mainboard	Physical 3D Printer Hardware	High: Incorrect G-code commands can cause physical mechanical collisions
`SupabaseStorageService.java`	`storeModelFile()`, `loadModelContent()`	Uploads and retrieves 3D files from cloud bucket	`ModelController`, `GenerationController`	Medium: Requires valid Supabase credentials or local cache

Chapter 17: Future Change & Modification Guide (Developer Handbook)

17.1 Step-by-Step Feature Implementation Protocol

Future software engineers or AI coding agents modifying PrintGuard AI must follow this rigorous change protocol:

1. **Identify Impact Radius:** Determine whether the change affects the Database (Flyway migration required), Backend (Spring Boot entity/controller), AI service (FastAPI endpoint), or Frontend (React component).
2. **Preserve Database Integrity:** Never run manual ALTER TABLE commands directly on production. Create a new numbered migration file in `backend-java/src/main/resources/db/migration/` (e.g., `V3__add_new_feature.sql`).
3. **Maintain Backward Compatibility:** Ensure older REST API response formats remain compatible with existing mobile or web clients.
4. **Execute Automated Tests:** Always run `./mvnw test` and `pytest` before committing any modifications.

5. **Update Documentation:** Reflect API, schema, or configuration variable updates in this Master Documentation.

17.2 Standard Reusable Change Request Template

```
### Change Request: [Brief Title]
1. Requested Change: Detailed description of the feature or bugfix.
2. Current Behavior: How the platform behaves prior to modification.
3. Expected Behavior: How the platform must behave post-modification.
4. Affected Components: [ ] Frontend  [ ] Backend  [ ] AI Service  [ ] Database  [ ] Slicer
5. Implementation Plan: Step-by-step code changes.
6. Database Migrations: SQL migration script (if schema altered).
7. Verification & Tests: Specific unit tests and manual steps executed.
8. Rollback Procedure: Commands to revert changes if regressions occur.
```

Chapter 18: Project Knowledge Summary (AI Agent Rapid Onboarding)

Rapid Context Summary for AI Developers & New Engineers

Project Name: PrintGuard AI 3D Printer Operating System
Primary Stack: Java 21 (Spring Boot 3.3.0), Python 3.11 (FastAPI, YOLOv8), React 18 (Vite, Three.js), PostgreSQL 16 (Flyway, Supabase RLS).
Key Entry Points: Backend: `PrintGuardApplication.java` | AI Service: `ai-service/app.py` | Frontend: `frontend/src/main.jsx`
Ports: Backend = `8080` | AI Service = `8001` | Frontend = `5173` | Database = `5432`
Hardware Control: In-browser W3C Web Serial API (115200/250000 baud) + OctoPrint REST API client.
Critical Invariant: Never expose database secrets or API keys. Always maintain watertight manifold geometry before sending models to PrusaSlicer.